

Secure Software Development Lifecycle (SSDLC)

Owner: Mark Stolte, Founder — markstolte02@gmail.com

Last reviewed: 2026-07-05

Stage: Pre-launch / active development

Purpose

Describes how a code or schema change moves from a laptop to production, and what checks run along the way. Written for the current solo-developer reality, with the checks that become required (rather than best-effort) once a second contributor joins.

Environment separation

- `EXPO_PUBLIC_DATA_SOURCE=mock` runs the app entirely against local prototype data (`src/services/mock-finance-service.ts`), with no network calls to Supabase or Plaid. This is the default while a feature is being built, so new UI and business logic can be exercised without touching a real backend or real financial data.
- `EXPO_PUBLIC_DATA_SOURCE=supabase` runs against a real Supabase project (`src/services/supabase-services.ts`). Sandbox Plaid credentials (`PLAID_ENV=sandbox`) are used for all development and testing; production Plaid credentials are requested from Plaid separately and are not used in local development.
- No real user's production financial data is used for development or debugging.

Source control and change flow

- Source is tracked in a private GitHub repository (`Mstolte02/penny-pilot`), with `main` as the default branch.
- Secrets are never committed: `.env` is git-ignored, and only `.env.example` (placeholder values) is tracked. Supabase Edge Function secrets are set via `supabase secrets set`, not via files in the repo.
- Database schema changes are made as versioned SQL files under `supabase/migrations/`, applied via the Supabase CLI, and reviewed for Row Level Security correctness before being applied to a shared project (every new user-data table must ship with an RLS policy in the same migration that creates it).
- Given the current team size, code review is currently self-review plus AI-assisted review (this project uses Claude Code, including its `/code-review` tooling) rather than a second human reviewer. A second human reviewer becomes a hard requirement for changes touching authentication, Row Level Security policies, or Plaid token handling as soon as a second contributor is available.

Pre-merge checks

Run before merging any change, currently manually (no CI pipeline is wired up yet — see "Planned" below):

- `npx tsc --noEmit` — the project is TypeScript-strict; this must be clean.
- `npm run lint (expo lint)` — ESLint via `eslint-config-expo`.
- Manual verification in the Expo web/dev-client preview for any UI-affecting change, including checking the browser console for runtime errors.

Planned: wire the above two checks into GitHub Actions (or equivalent CI) as a required status check on `main`, so they run automatically instead of relying on the developer to remember. Tracked as an open item rather than claimed as already in place.

Dependency management

- Dependencies are pinned via `package-lock.json` and managed through `npm`.
- The Expo SDK is upgraded deliberately (the project currently targets SDK 56) rather than auto-upgraded, since Expo SDK bumps can change native behavior.
- See [Vulnerability Management](#) for how known-vulnerable dependencies are tracked.

Secure coding practices specific to this codebase

- Row Level Security is the primary authorization boundary for user data — application code must not assume it is the only thing preventing cross-user data access, and new tables must not skip it.
- Anything that talks to Plaid directly (the client secret, the token-encryption key, access-token decryption) lives only in Supabase Edge Functions (`supabase/functions/plaid-*`, `supabase/functions/_shared/plaid.ts`), never in client-side code.
- Only `EXPO_PUBLIC_*`-prefixed environment variables are safe to reference from the mobile app, since Expo inlines them into the client bundle at build time (`docs/technical-architecture.md`). Anything else (Plaid secret, AI provider keys, the token-encryption key) must be read only inside a backend function.

Testing

The project currently relies on type-checking, linting, and manual/browser-preview verification rather than an automated test suite. Adding automated tests (particularly for `src/domain` finance calculations and the CSV import parser) is a planned improvement, not yet in place.

